

Predicting Movie Ratings Using Film Features

DSA 210 – Introduction to Data Science | 2025–2026 Spring Term

Yağız Mert Göçmen

1. Motivation

The film industry produces enormous amounts of data, yet predicting how audiences will respond to a movie remains a genuinely difficult problem. IMDb and TMDb ratings are among the most widely referenced measures of movie quality and audience satisfaction — but what actually drives those numbers?

The goal of this project is to explore whether objective, measurable film features — budget, genre, runtime, vote count, revenue — can be used to predict a movie’s audience rating. It’s a practically interesting question and a technically approachable one, which made it a good fit for the data science pipeline covered in this course.

2. Data Source

The dataset used is the **TMDb 5000 Movie Dataset**, obtained from Kaggle. It consists of two CSV files:

- `tmdb_5000_movies.csv` — budget, revenue, genres, runtime, release date, vote average, vote count, popularity, keywords, production companies
- `tmdb_5000_credits.csv` — cast and crew information (director extracted)

The two files were merged on a shared movie ID column. TMDb’s `vote_average` field (audience ratings on a 1–10 scale) is used as the target variable throughout the analysis. After cleaning, the final dataset contains **4,274 movies**.

3. Data Analysis

3.1 Data Cleaning & Preprocessing

The raw data needed quite a bit of work before it was usable:

- JSON-encoded columns (genres, cast, crew) were parsed to extract names and the director
- Budget and revenue entries recorded as zero were replaced with NaN — zero almost certainly means missing data, not an actual value
- Rows with unreleased status, fewer than 10 votes, missing runtime, or zero rating were removed
- Log transformations were applied to budget, revenue, and vote count to reduce skewness

- Outliers were removed using the IQR method (multiplier = 3.0) on runtime, vote average, and popularity

This reduced the dataset from 4,803 to 4,274 rows.

3.2 Exploratory Data Analysis

The first thing worth checking is the shape of the target variable. As Figure 1 shows, ratings follow a roughly normal distribution centered around 6.2 (mean 6.21, median 6.30), with a slight left skew. Most movies cluster in the 5.5–7.5 range, and very few fall below 4 or above 8.

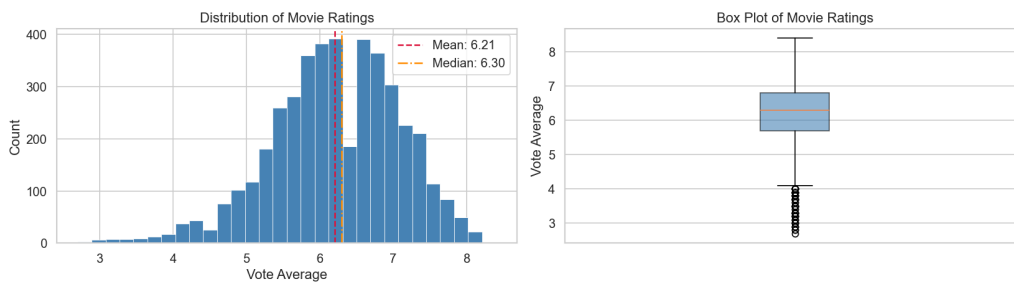


Figure 1: Distribution of movie ratings (left: histogram with mean/median; right: box plot).

Figure 2 shows how average ratings and movie counts evolved over time. Ratings have been slowly declining since the 1960s, likely as the volume of films increased and the sample became less selective.

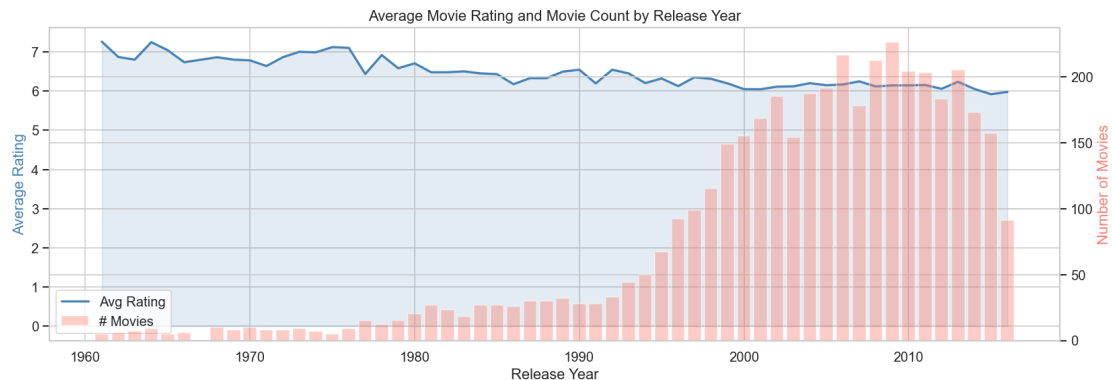


Figure 2: Average movie rating (blue line) and number of movies released (pink bars) by year.

Figure 3 breaks ratings down by primary genre. Western, Documentary, and Drama sit at the top; Horror and Action at the bottom. The spread within each genre is quite large, which already hints that genre alone won't be enough to predict ratings reliably.

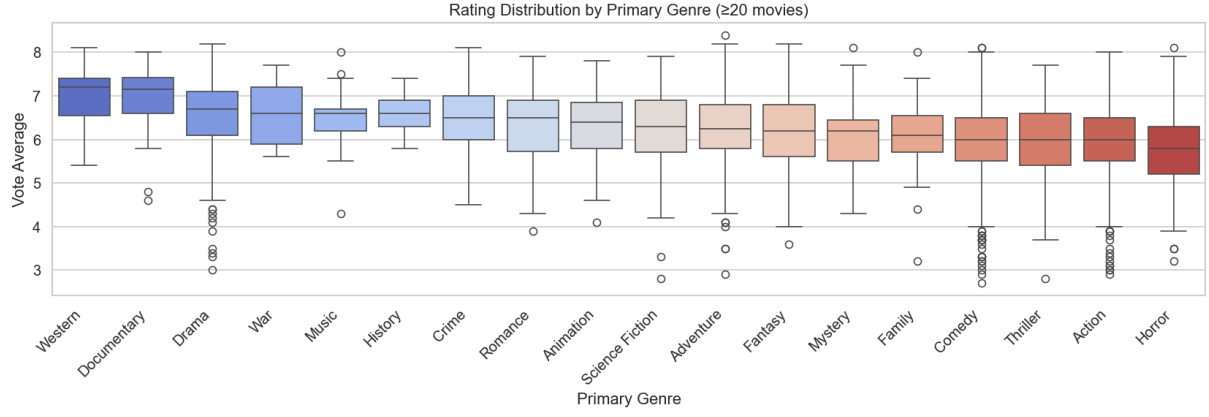


Figure 3: Rating distribution by primary genre (genres with at least 20 films shown).

Figure 4 shows how the four main numerical features relate to ratings. Vote count and runtime show the clearest positive trends; budget has a slight negative relationship, which is already interesting given Hypothesis H2.

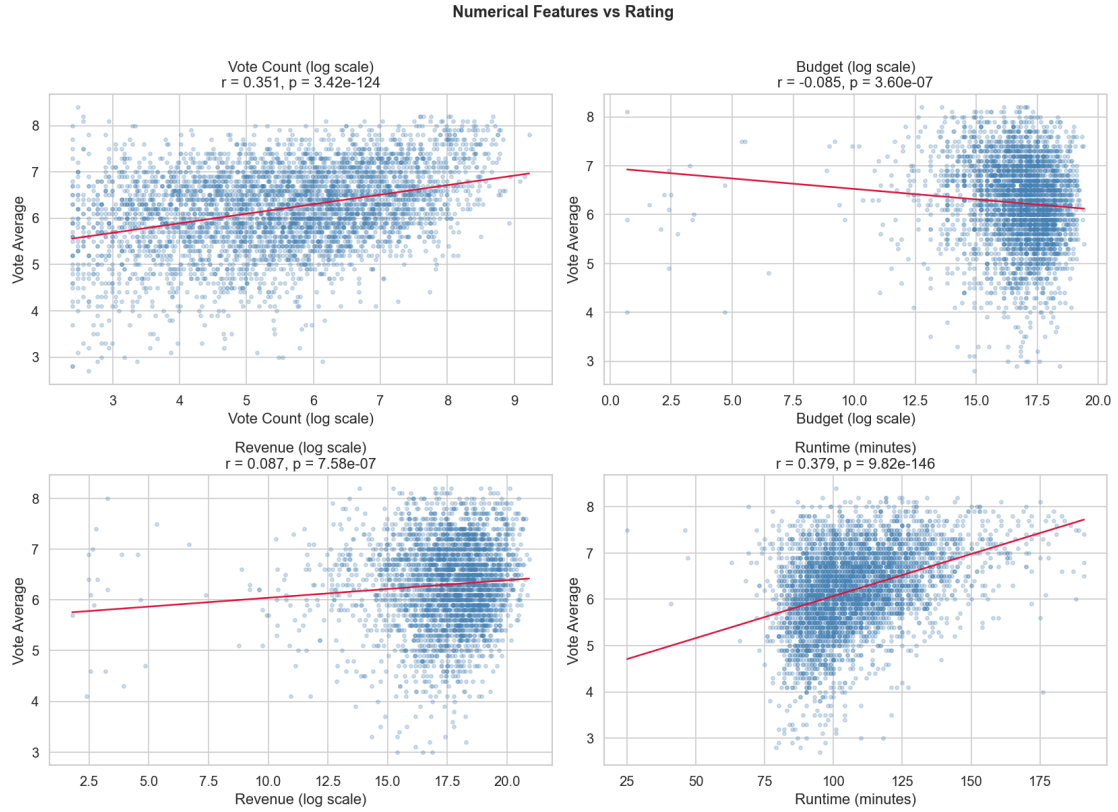


Figure 4: Scatter plots of numerical features vs. vote average, with Pearson correlation coefficients.

Figure 5 summarizes pairwise correlations between numerical features. Runtime ($r = 0.38$) and `log_votes` ($r = 0.35$) are the strongest predictors of rating. Budget has a slight negative correlation ($r = -0.09$), while revenue is near zero. Notably, `log_votes` and popularity are strongly correlated ($r = 0.85$), which suggests some redundancy in those features.

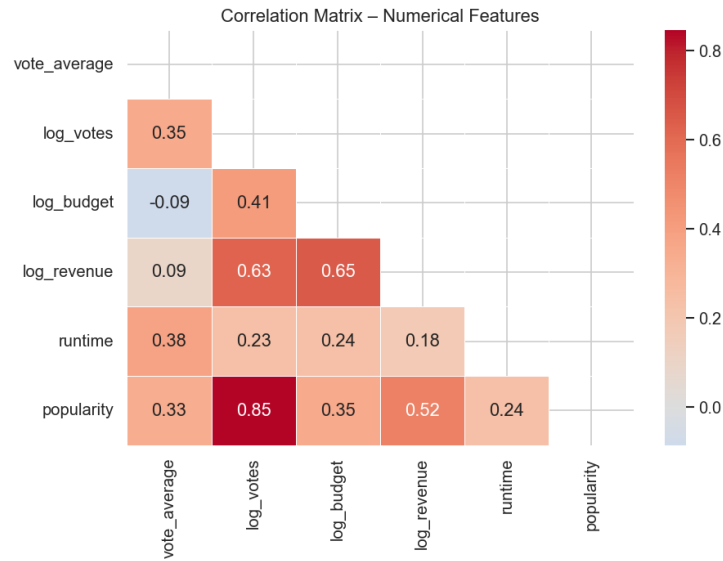


Figure 5: Correlation matrix for numerical features.

Figure 6 shows rating distributions split into budget quartiles. The downward trend from low to high budget is visible, though the effect is modest and there is substantial overlap.

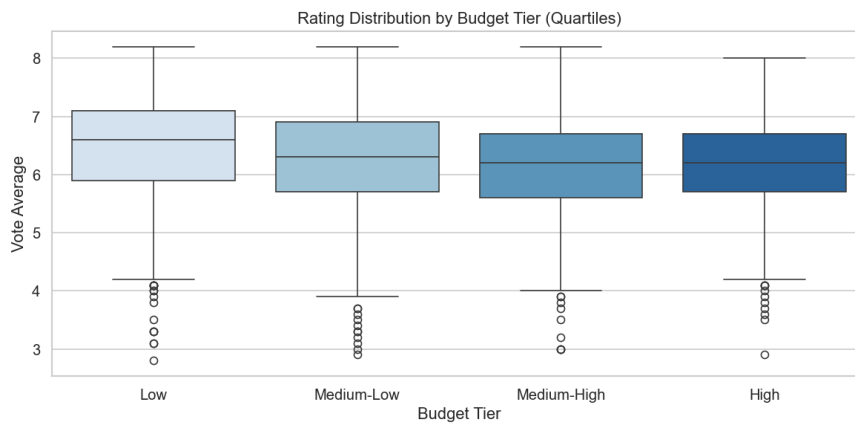


Figure 6: Rating distribution by budget tier (quartile split).

Figure 7 illustrates the language imbalance in the dataset (English dominates at roughly 94%) and the rating gap — non-English films consistently score higher across almost every language group.

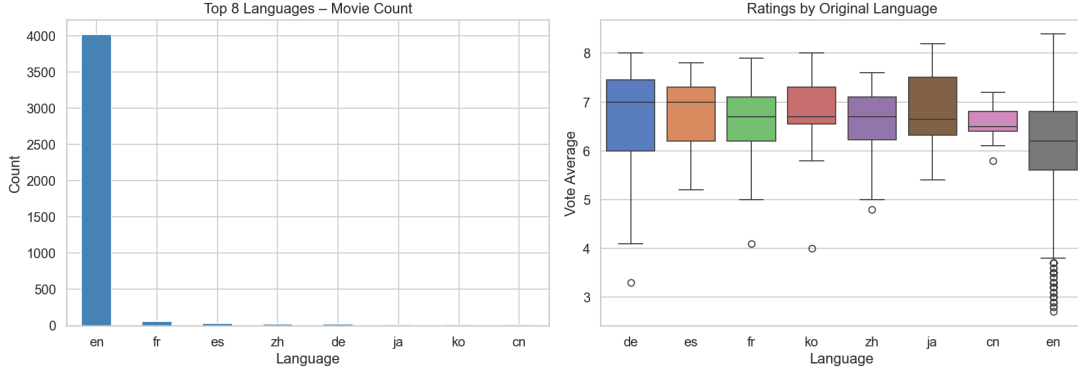


Figure 7: Movie count by language (left) and rating distribution by language (right).

3.3 Hypothesis Testing

Four hypotheses were tested at $\alpha = 0.05$:

- **H1 – Vote count is positively correlated with rating:** Pearson $r = 0.35$, $p \approx 0 \rightarrow$ Rejected H_0 . Confirmed positive correlation.
- **H2 – High-budget films are rated differently from low-budget films:** Welch’s t -test, $p \approx 8 \times 10^{-12} \rightarrow$ Rejected H_0 . Somewhat surprisingly, low-budget films (mean 6.42) are rated slightly higher than high-budget ones (mean 6.14).
- **H3 – Genre has a significant effect on rating:** One-way ANOVA $F = 32.0$, $p \approx 0 \rightarrow$ Rejected H_0 . Tukey HSD confirmed multiple significant pairwise differences.
- **H4 – English vs. non-English ratings:** Welch’s t -test, $p \approx 1.6 \times 10^{-21} \rightarrow$ Rejected H_0 . Non-English films are rated significantly higher, which was actually the opposite of the initial expectation.

3.4 Machine Learning

Seven regression models were trained on an 80/20 train-test split to predict `vote_average`. Features used: `log_votes`, `log_budget`, `log_revenue`, `runtime`, `popularity`, `release_year`, primary genre (one-hot), and language group (one-hot). Missing numerical values were imputed with the column median.

Table 1: Model performance on the test set.

Model	RMSE	R ²
Gradient Boosting	0.6503	0.4284
Random Forest	0.6576	0.4154
Ridge	0.6763	0.3817
Linear Regression	0.6766	0.3811
Lasso	0.6785	0.3777
XGBoost	0.6918	0.3530
Decision Tree	0.7475	0.2448

Figure 8 makes the ranking easier to read visually. Gradient Boosting and Random Forest are clearly ahead; the Decision Tree trails by a noticeable margin.

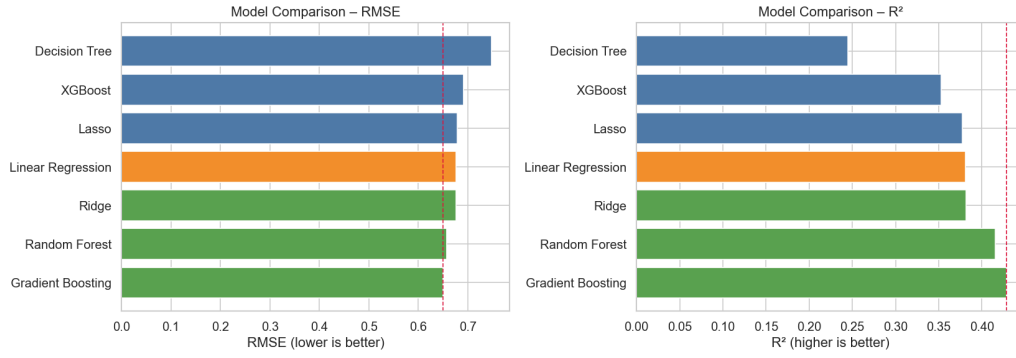


Figure 8: Model comparison by RMSE (lower is better) and R^2 (higher is better).

Figure 9 shows actual vs. predicted values for all seven models. The ensemble methods (Random Forest, Gradient Boosting) cluster more tightly around the diagonal. The Decision Tree's characteristic banding pattern is clearly visible — it can only output a limited set of discrete values.

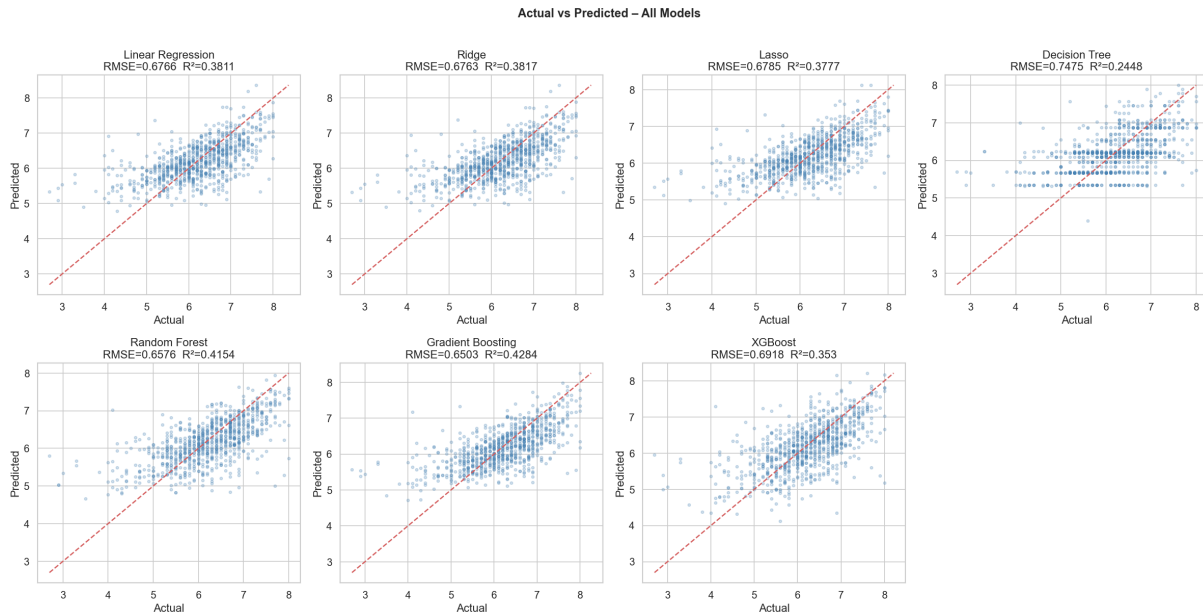


Figure 9: Actual vs. predicted ratings for all models. The dashed line represents perfect prediction.

Figure 10 shows residual plots. Linear models and ensemble methods display a roughly random scatter around zero — a good sign. The Decision Tree again stands out with its banded residuals, confirming that it's underfitting.

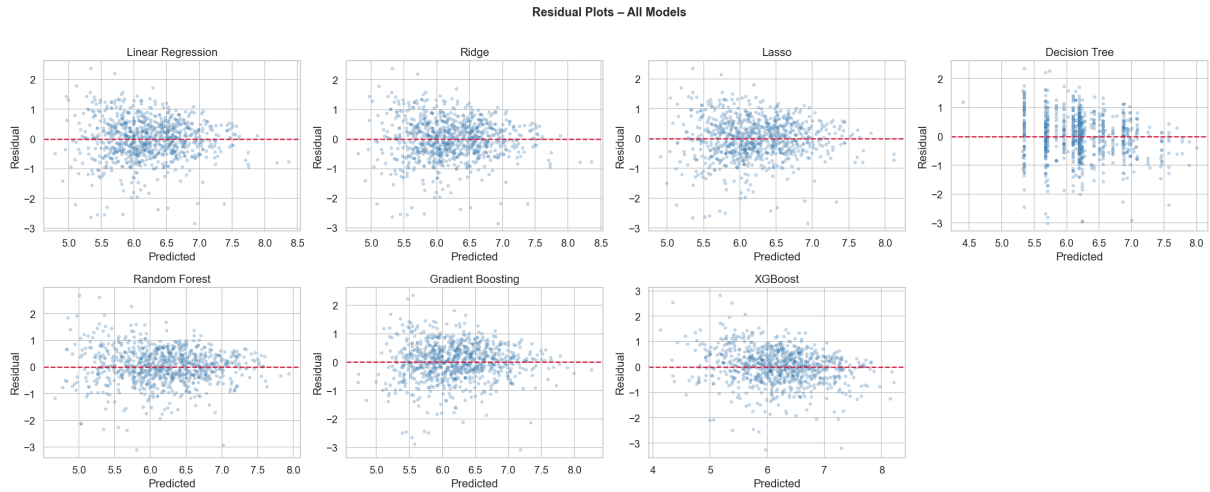


Figure 10: Residual plots for all models. Ideal behavior is a random scatter around the zero line.

Figure 11 shows feature importance for the four tree models. `log_votes` and `runtime` dominate in Gradient Boosting and Random Forest. XGBoost gives more weight to genre and language features, which may explain its slightly weaker performance on this dataset.

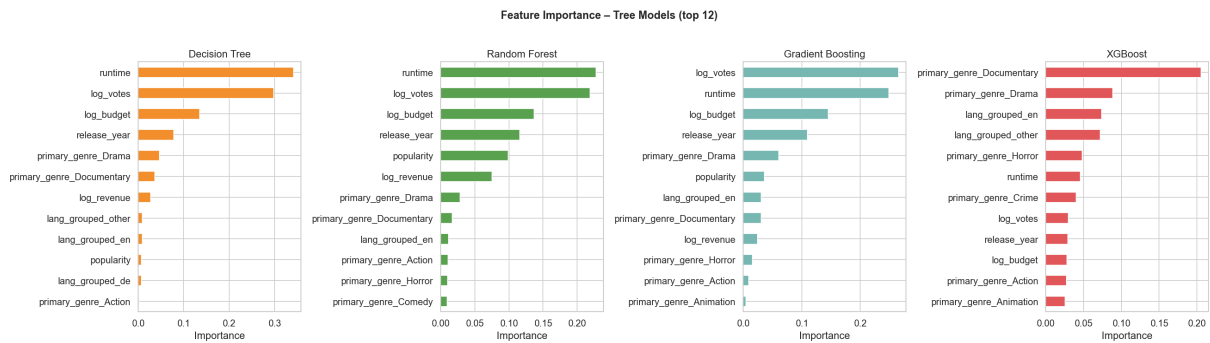


Figure 11: Feature importance (top 12) for tree-based models.

Figure 12 shows coefficients for the three linear models. Plain Linear Regression produces wildly inflated genre coefficients (note the 10^{12} scale), a sign of multicollinearity. Ridge and Lasso regularization fix this — their coefficients are stable and interpretable, with `log_votes` and `runtime` coming out on top.

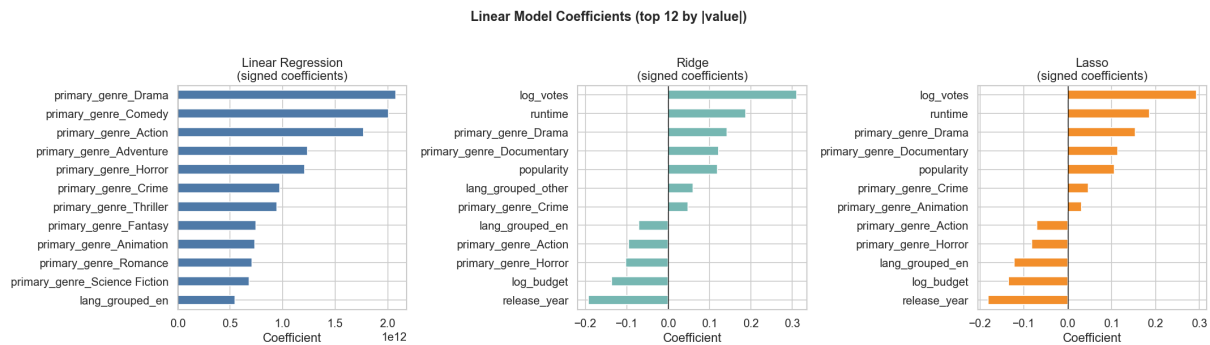


Figure 12: Top 12 coefficients by absolute value for the three linear models.

5-fold cross-validation on the top three tree models confirmed the rankings: Gradient Boosting (mean RMSE 0.670), Random Forest (0.683), XGBoost (0.709).

4. Findings

- **Best model:** Gradient Boosting with $\text{RMSE} = 0.65$ and $R^2 = 0.43$. Predictions are off by less than a point on average, which feels reasonable given the features available.
- **Most important feature:** `log_votes` was the top feature across all tree models, consistent with H1. Films that attract more votes tend to be the ones people genuinely care about.
- **Linear models are competitive:** Ridge and Linear Regression both hit $R^2 \approx 0.38$, not far behind the ensemble methods. The feature-rating relationships are largely linear.
- **Ensemble beats single tree:** Decision Tree alone scored $R^2 = 0.24$, while Random Forest and Gradient Boosting reached 0.41–0.43 — a pretty clear gap.
- **Budget paradox:** Low-budget films are rated slightly higher on average. Big-budget productions may be optimized for commercial appeal rather than quality, and audiences seem to reflect that.
- **Non-English films:** Only 6% of the dataset, but consistently rated higher. This is likely a selection effect — foreign films only accumulate enough votes to appear in the dataset if they were already well-regarded internationally.

5. Limitations & Future Work

- `log_votes` is the strongest predictor but isn't available before release, so the model can't really be used for pre-release predictions — which would have been the most useful application.
 - Budget and revenue data was missing for roughly 25% of movies, which likely limited model performance.
 - No hyperparameter tuning was done — all models ran on defaults. GridSearchCV or RandomizedSearchCV could push results higher, especially for XGBoost.
 - The biggest gap is probably unmeasured quality factors: screenplay, acting, direction. $R^2 = 0.43$ means 57% of the variance is still unexplained, and most of that is probably things that are just hard to quantify.
 - The dataset is 94% English-language films, so generalization to world cinema is limited.
 - Future extensions could include NLP features from movie overviews, director/actor historical rating averages, or social media sentiment at release time.
-

AI Tool Disclosure: Claude (Anthropic) was used as an AI assistant throughout this project for tasks including code writing, debugging, and report drafting. All outputs were reviewed, tested, and adapted by the author.